

SIMATS ENGINEERING

CO2 ASSESSMENT TOOL 1

HITESH C

192524005

Course Code: CSA1401

Course Name: Compiler Design

Question 1 — LL (1) Predictive Parser Design

Grammar:

```
S → A B D
A → a | D b | ε
B → g D | d A | ε
D → e | f
```

(i) FIRST and FOLLOW Sets

Step 1: FIRST(D). D has only terminal-starting productions, so $\text{FIRST}(D) = \{e, f\}$.

Step 2: FIRST(A). $A \rightarrow a$ contributes 'a'. $A \rightarrow Db$ contributes $\text{FIRST}(D) = \{e, f\}$. $A \rightarrow \epsilon$ contributes ϵ . Hence $\text{FIRST}(A) = \{a, e, f, \epsilon\}$.

Step 3: FIRST(B). $B \rightarrow gD$ contributes 'g'. $B \rightarrow dA$ contributes 'd'. $B \rightarrow \epsilon$ contributes ϵ . Hence $\text{FIRST}(B) = \{d, g, \epsilon\}$.

Step 4: FIRST(S). $S \rightarrow A B D$. Since A can derive ϵ , FIRST(S) starts with $(\text{FIRST}(A) - \epsilon) \cup (\text{FIRST}(B) - \epsilon)$. Since B can also derive ϵ , FIRST(D) is added as well. Since D cannot derive ϵ , the chain stops there and ϵ is NOT included. Hence $\text{FIRST}(S) = \{a, d, e, f, g\}$.

Non-terminal	FIRST set
S	$\{a, d, e, f, g\}$
A	$\{a, e, f, \epsilon\}$
B	$\{d, g, \epsilon\}$
D	$\{e, f\}$

Step 5: FOLLOW(S) = $\{\$ \}$, since S is the start symbol.

Step 6: FOLLOW(B). B occurs only in $S \rightarrow A B D$, followed by D. $\text{FIRST}(D) = \{e, f\}$ and D cannot derive ϵ , so $\text{FOLLOW}(B) = \{e, f\}$.

Step 7: FOLLOW(A). A occurs in $S \rightarrow A B D$ (followed by BD) and in $B \rightarrow d A$ (at the end). From $S \rightarrow A B D$: $FIRST(BD) = (FIRST(B) - \epsilon) \cup FIRST(D) = \{d, g\} \cup \{e, f\} = \{d, g, e, f\}$ (added to FOLLOW(A)). From $B \rightarrow dA$: A is at the end of the production, so $FOLLOW(B) = \{e, f\}$ is also added. Hence $FOLLOW(A) = \{d, e, f, g\}$.

Step 8: FOLLOW(D). D occurs in $S \rightarrow ABD$ (at the end $\rightarrow$ add $FOLLOW(S) = \{\$ \}$), in $A \rightarrow Db$ (followed by 'b' $\rightarrow$ add $\{b\}$), and in $B \rightarrow gD$ (at the end $\rightarrow$ add $FOLLOW(B) = \{e, f\}$). Hence $FOLLOW(D) = \{\$, b, e, f\}$.

Non-terminal	FOLLOW set
S	$\{\$ \}$
A	$\{d, e, f, g\}$
B	$\{e, f\}$
D	$\{\$, b, e, f\}$

(ii) Predictive Parsing Table

For each production $A \rightarrow \alpha$, entries are placed at $M[A, t]$ for every terminal $t \in FIRST(\alpha)$; if $\epsilon \in FIRST(\alpha)$, entries are also placed at $M[A, t]$ for every $t \in FOLLOW(A)$.

NT	a	b	d	e	f	g	\$
S	$S \rightarrow ABD$	—	$S \rightarrow ABD$	$S \rightarrow ABD$	$S \rightarrow ABD$	$S \rightarrow ABD$	—
A	$A \rightarrow a$	—	$A \rightarrow \epsilon$	$A \rightarrow Db /$ $A \rightarrow \epsilon$ (CONFLICT)	$A \rightarrow Db /$ $A \rightarrow \epsilon$ (CONFLICT)	$A \rightarrow \epsilon$	—
B	—	—	$B \rightarrow dA$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	$B \rightarrow gD$	—
D	—	—	—	$D \rightarrow e$	$D \rightarrow f$	—	—

(iii) Verification of the LL(1) Property

FIRST/FIRST conflict analysis: Within a single non-terminal, alternative productions must have disjoint FIRST sets. For A: $\text{FIRST}(a) = \{a\}$, $\text{FIRST}(Db) = \{e, f\}$, and ϵ is separate. $\{a\} \cap \{e, f\} = \emptyset$, so there is no FIRST/FIRST conflict. For B and S the alternatives are likewise pairwise disjoint. Hence no FIRST/FIRST conflicts exist.

FIRST/FOLLOW conflict analysis for ϵ -productions: A rule $N \rightarrow \epsilon$ conflicts with another rule $N \rightarrow \alpha$ if $(\text{FIRST}(\alpha) - \epsilon) \cap \text{FOLLOW}(N) \neq \emptyset$. For B: $(\text{FIRST}(gD|dA) = \{d, g\}) \cap \text{FOLLOW}(B) = \{e, f\} = \emptyset$ — no conflict. For A: $(\text{FIRST}(Db) = \{e, f\}) \cap \text{FOLLOW}(A) = \{d, e, f, g\} = \{e, f\} \neq \emptyset$. This is a genuine FIRST/FOLLOW conflict.

Identification of conflicting entries: The parsing table cells $M[A, e]$ and $M[A, f]$ each contain two productions ($A \rightarrow Db$ and $A \rightarrow \epsilon$), which is not permitted in a deterministic table-driven LL(1) parser.

Final conclusion: Because cells $M[A, e]$ and $M[A, f]$ each hold more than one production, the grammar is NOT LL(1).

(iv) Demonstration of Predictive Parsing

Consider the valid string “ae”, derived as $S \Rightarrow ABD \Rightarrow aBD$ (choosing $A \rightarrow a$) $\Rightarrow aD$ (choosing $B \rightarrow \epsilon$) $\Rightarrow ae$ (choosing $D \rightarrow e$). At the point where A is expanded there is no conflict (lookahead 'a' uniquely selects $A \rightarrow a$), so the table can still be driven deterministically for this particular string.

Step	Stack	Input	Action
1	\$ S	a e \$	$M[S, a] = S \rightarrow ABD$; push D, B, A
2	\$ D B A	a e \$	$M[A, a] = A \rightarrow a$; push a
3	\$ D B a	a e \$	match 'a'; pop, advance input

Step	Stack	Input	Action
4	\$ D B	e \$	$M[B,e] = B \rightarrow \epsilon$; pop B
5	\$ D	e \$	$M[D,e] = D \rightarrow e$; push e
6	\$ e	e \$	match 'e'; pop, advance input
7	\$	\$	stack and input both empty $\rightarrow$ ACCEPT

The string “ae” is therefore accepted.

(v) Design Recommendations

- The grammar cannot be used directly for table-driven LL(1) parsing, since the predictive parsing table contains multiply-defined entries at $M[A,e]$ and $M[A,f]$.
- Cause of the conflict: production $A \rightarrow Db$ introduces $\{e, f\}$ into $FIRST(A)$, while the same symbols $\{e, f\}$ reach $FOLLOW(A)$ indirectly through $S \rightarrow ABD$ together with $B \rightarrow \epsilon$ (whatever follows B, namely $FIRST(D) = \{e, f\}$, also follows A). This indirect propagation through a nullable B is the root cause.
- Suggested modification: eliminate the ambiguity by restructuring the grammar so that D no longer follows a nullable B directly after A — for example, by left-factoring A's alternatives or by merging B and D into new non-terminals that keep $FIRST(A)$ and $FOLLOW(A)$ disjoint. Alternatively, the ambiguity could be resolved with additional lookahead (an LL(2) parser) or by switching to a bottom-up (LR-family) parser, which tolerates such conflicts far more gracefully.
- Advantages of a predictive parser: single left-to-right scan with no backtracking, linear-time $O(n)$ parsing, simple table-driven or recursive-descent implementation, precise and early syntax-error detection with clear

expected-symbol diagnostics, and natural integration with syntax-directed translation during parsing.

Question 2 — Shift-Reduce Parser for Arithmetic Expressions

Grammar:

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid id$$

Input string: $id * id / (id + id)$

(i) Stack-Based Shift-Reduce Parser Design

The parser maintains a stack (initialised with the end marker $\$$) and an input buffer terminated by $\$$. At each step it compares the topmost stack symbols with the next input symbol: it shifts the next input symbol onto the stack when no complete right-hand side (handle) is yet on top, and it reduces the handle at the top of the stack to its left-hand side when precedence and associativity rules indicate that the current operator has already bound its operands. Parsing halts with ACCEPT when the stack contains only $\$ E$ and the input is exhausted, or with an error if neither shift nor reduce is applicable.

(ii)–(iii), (v) Parsing Trace, Handles and Reductions

Handle-recognition rule used: $*$ and $/$ bind tighter than nothing else here (only one precedence class besides parentheses) and are left-associative, so when the stack shows $E * E$ (or E / E) and the next symbol has equal precedence, the pending $E * E / E / E$ is reduced first; likewise a completed $E + E$ or $E(\dots)$ is reduced as soon as the next symbol cannot extend it further (a lower-precedence operator, $)$, or $\$$).

Step	Stack	Remaining Input	Action
1	$\$$	$id * id / (id + id) \$$	Shift id
2	$\$ id$	$* id / (id + id) \$$	Reduce $id \rightarrow E$ (handle: id)
3	$\$ E$	$* id / (id + id) \$$	Shift $*$
4	$\$ E *$	$id / (id + id) \$$	Shift id

Step	Stack	Remaining Input	Action
5	\$ E * id	/ (id + id) \$	Reduce id $\rightarrow$ E (handle: id)
6	\$ E * E	/ (id + id) \$	Reduce E * E $\rightarrow$ E (handle: E * E; '/' has equal precedence, left- assoc $\Rightarrow$ reduce first)
7	\$ E	/ (id + id) \$	Shift /
8	\$ E /	(id + id) \$	Shift (
9	\$ E / (	id + id) \$	Shift id
10	\$ E / (id	+ id) \$	Reduce id $\rightarrow$ E (handle: id)
11	\$ E / (E	+ id) \$	Shift +
12	\$ E / (E +	id) \$	Shift id
13	\$ E / (E + id	) \$	Reduce id $\rightarrow$ E (handle: id)
14	\$ E / (E + E	) \$	Reduce E + E $\rightarrow$ E (handle: E + E; ')' cannot extend it)
15	\$ E / (E	) \$	Shift)
16	\$ E / (E)	\$	Reduce (E) $\rightarrow$ E (handle: (E))
17	\$ E / E	\$	Reduce E / E $\rightarrow$ E (handle: E / E)
18	\$ E	\$	ACCEPT

(iv) Parser Decisions and Shift-Reduce Conflicts

Because the grammar $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid id$ is ambiguous, a naive bottom-up parser meets a genuine shift/reduce conflict at every point where E op E sits on top of the stack and another operator follows: it could reduce the visible E op E, or shift the new operator expecting a larger right operand. These conflicts are

resolved with an operator-precedence and associativity table: $*$ and $/$ outrank $+$ and $-$, so when $E * E$ (or E / E) is on top and the lookahead is $+$ or $-$ the parser reduces; if the lookahead is another $*$ or $/$ (equal precedence, left-associative) the parser also reduces rather than shifts, ensuring left-to-right grouping; parentheses always shift until the matching $)$ triggers a reduction of (E) to E , which is treated as having the highest precedence of all. Step 6 and step 14 above are the concrete instances where the conflict was resolved by 'reduce' rather than 'shift'.

Verification: with the input fully consumed and the stack reduced to $\$ E$, the string $id * id / (id + id)$ is successfully accepted by the parser.

Question 3 — Recursive Descent Parser

Grammar:

```
S → ( L ) | a
L → S L'
L' → , S L' | ε
```

Input string: (a)

(i)–(ii) Recursive Procedures (Predictive, No Backtracking)

$FIRST(S) = \{ (, a \}$. Inside procedure S, the current lookahead uniquely decides between the two alternatives, so no backtracking is needed. $FIRST(L') = \{ , \}$ with ϵ as a valid choice when the lookahead is ')' ($\in FOLLOW(L')$).

```
procedure S():
    if lookahead == '(' :
        match('(')
        L()
        match(')')
    else if lookahead == 'a' :
        match('a')
    else :
        error("expected '(' or 'a'")

procedure L():
    S()
    Lprime()

procedure Lprime():
    if lookahead == ',' :
        match(',')
        S()
        Lprime()
    else :
```

```

        return                //  $\epsilon$ -production, valid when
lookahead  $\in \{ ' ) ' , \$ \}$ 

procedure match(expected) :
    if lookahead == expected :
        lookahead = nextToken()
    else :
        error("expected '" + expected + "' but found '"
+ lookahead + "'")
        recover()            // panic-mode: see part (v)

```

(iii) Sequence of Calls and Terminal Matching for '(a)'

Call / Action	Lookahead	Remaining Input	Effect
S()	(	(a)	'(' seen $\rightarrow$ match '(' , call L()
match('(')	(	(a)	consumes '(' , advances to 'a'
L() $\rightarrow$ S()	a	a)	'a' seen $\rightarrow$ match 'a'
match('a')	a	a)	consumes 'a' , advances to ')'
L() $\rightarrow$ L'()	)	)	lookahead ')' $\notin \{ , \}$ $\rightarrow \epsilon$ -production, return
S() $\rightarrow$ match(')')	)	)	consumes ')' , advances to end-of-input

(iv) Acceptance / Rejection

Leftmost derivation: $S \Rightarrow (L) \Rightarrow (S L') \Rightarrow (a L') \Rightarrow (a \epsilon) \Rightarrow (a)$. This matches the input string “(a)” exactly, and every call returns without triggering error(); since the input is fully consumed exactly when S() returns to the top level, the string “(a)” is ACCEPTED.

(v) Error Detection and Recovery

Whenever `match()` finds a mismatch between the expected terminal and the actual lookahead, it reports a diagnostic naming both the expected and found symbols. Rather than aborting immediately, the parser performs panic-mode recovery: it discards input tokens until it reaches a synchronizing token drawn from a pre-agreed set (typically ')', ';', or end-of-input for this grammar), then resumes normal calls from the enclosing procedure. This allows a single parse to report multiple independent errors instead of stopping at the first one, at the cost of not always pinpointing the precise error location.

Question 4 — Operator Precedence Parser

Input string: id - id * id

(i) Unambiguous Operator Grammar

An operator grammar must have no ϵ -productions and no two adjacent non-terminals on any right-hand side. The following grammar for +, -, *, (,) satisfies this requirement while remaining a natural operator grammar suitable for precedence-relation parsing:

$E \rightarrow E + E \mid E - E \mid E * E \mid (E) \mid id$

(ii)–(iii) Precedence Relations and Operator Precedence Table

Convention: $a <\cdot b$ means a yields precedence to b (shift); $a \cdot >b$ means a takes precedence over b (reduce the handle ending at a); $a =\cdot b$ applies only to matching parenthesis pairs; a blank cell (—) denotes an undefined / error relation. * binds tighter than + and -; + and - share the same precedence; () has the highest precedence until matched; all operators are left-associative.

	id	+	-	*	(	)	\$
id	—	$\cdot >$	$\cdot >$	$\cdot >$	—	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
-	$< \cdot$	$\cdot >$	$\cdot >$	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$	$< \cdot$	$\cdot >$	$\cdot >$
(	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$= \cdot$	—
)	—	$\cdot >$	$\cdot >$	$\cdot >$	—	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	$< \cdot$	—	accept

(Row symbol = topmost terminal on stack; column symbol = next input symbol.)

(iv) Stack-Based Parsing Algorithm

- Initialise the stack with \$.
- Let a be the topmost terminal on the stack (skipping any non-terminal directly beneath it) and b the next input symbol.
- If $a < \cdot b$ or $a = \cdot b$: shift b onto the stack and advance the input.
- If $a \cdot > b$: pop the handle — everything above the nearest stack terminal that has a $< \cdot$ relation with the terminal just below the handle — and replace it with the corresponding non-terminal E.
- If no relation is defined between a and b : report a syntax error.
- Stop with ACCEPT when the stack is \$ E and the input is \$.

(v) Step-by-Step Trace for 'id - id * id'

Step	Stack	Input	Relation	Action
1	\$	id - id * id \$	$\$ < \cdot \text{id}$	Shift id
2	\$ id	- id * id \$	$\text{id} \cdot > -$	Reduce $\text{id} \rightarrow E$ (handle: id)
3	\$ E	- id * id \$	$\$ < \cdot -$	Shift -
4	\$ E -	id * id \$	$- < \cdot \text{id}$	Shift id
5	\$ E - id	* id \$	$\text{id} \cdot > *$	Reduce $\text{id} \rightarrow E$ (handle: id)
6	\$ E - E	* id \$	$- < \cdot *$	Shift *
7	\$ E - E *	id \$	$* < \cdot \text{id}$	Shift id
8	\$ E - E * id	\$	$\text{id} \cdot > \$$	Reduce $\text{id} \rightarrow E$ (handle: id)
9	\$ E - E * E	\$	$* \cdot > \$$	Reduce $E * E \rightarrow E$ (handle: E * E)
10	\$ E - E	\$	$- \cdot > \$$	Reduce $E - E \rightarrow E$ (handle: E - E)
11	\$ E	\$	—	ACCEPT

Because '-' yields precedence to '*' (step 6), the multiplication is fully reduced (step 9) before the subtraction is reduced (step 10), correctly reflecting that * binds tighter than -. The input $\text{id} - \text{id} * \text{id}$ is accepted.

Question 5 — LALR (1) Parser Construction

Grammar:

```
S → A A
A → a A | b
```

(i) Augmented Grammar

```
(0) S' → S
(1) S → A A
(2) A → a A
(3) A → b
```

(iii) FIRST and FOLLOW Sets

Non-terminal	FIRST	FOLLOW
S'	{a, b}	{ $\$$ }
S	{a, b}	{ $\$$ }
A	{a, b}	{a, b, $\$$ }

$\text{FOLLOW}(A) = \{a, b, \$\}$: the first A in $S \rightarrow AA$ is followed by the second A, contributing $\text{FIRST}(A) = \{a, b\}$; the second A is at the end of $S \rightarrow AA$, contributing $\text{FOLLOW}(S) = \{\$ \}$. Also, $A \rightarrow aA$ places A at the end of its own production, contributing $\text{FOLLOW}(A)$ back to itself (already accounted for).

(ii) Canonical Collection of LR(1) Item Sets

Closure and GOTO computations yield the following canonical LR(1) states (lookaheads shown after the comma):

```
I0 = { [S' → · S, $], [S → · AA, $], [A → · aA, a/b], [A → · b, a/b] }
I1 = GOTO(I0, S) = { [S' → S ·, $] }
    (ACCEPT state)
I2 = GOTO(I0, A) = { [S → A · A, $], [A → · aA, $], [A → · b, $] }
I3 = GOTO(I0, a) = { [A → a · A, a/b], [A → · aA, a/b],
    [A → · b, a/b] }
```



```

I4 = GOTO(I0,b) = { [A→b·,a/b] }
I5 = GOTO(I2,A) = { [S→AA·,$] }
I6 = GOTO(I2,a) = { [A→a·A,$], [A→·aA,$], [A→·b,$] }
I7 = GOTO(I2,b) = { [A→b·,$] }
I8 = GOTO(I3,A) = { [A→aA·,a/b] }
    GOTO(I3,a) = I3 (self-loop); GOTO(I3,b) = I4
(same core+lookahead as I4)
I10 = GOTO(I6,A) = { [A→aA·,$] }
    GOTO(I6,a) = I6 (self-loop); GOTO(I6,b) = I7
(same core+lookahead as I7)

```

(iv) Merging States with Identical Cores — LALR(1) Table

I3 and I6 share the identical item core $\{[A \rightarrow a \cdot A], [A \rightarrow \cdot aA], [A \rightarrow \cdot b]\}$, differing only in lookahead ($\{a,b\}$ vs $\{\$ \}$); they merge into one LALR state carrying the union $\{a, b, \$\}$. Likewise I4/I7 merge (core $\{[A \rightarrow b \cdot]\}$) and I8/I10 merge (core $\{[A \rightarrow aA \cdot]\}$). Re-checking each merged state shows no new shift/reduce or reduce/reduce conflicts are introduced, so the grammar is confirmed LALR(1).

Final LALR(1) states (renumbered):

State	Items
0	$[S' \rightarrow \cdot S, \$], [S \rightarrow \cdot AA, \$], [A \rightarrow \cdot aA, a/b], [A \rightarrow \cdot b, a/b]$
1	$[S' \rightarrow S \cdot, \$]$ — ACCEPT
2	$[S \rightarrow A \cdot A, \$], [A \rightarrow \cdot aA, \$], [A \rightarrow \cdot b, \$]$
3	$[A \rightarrow a \cdot A, a/b/\$], [A \rightarrow \cdot aA, a/b/\$], [A \rightarrow \cdot b, a/b/\$]$ (merged I3+I6)
4	$[A \rightarrow b \cdot, a/b/\$]$ (merged I4+I7)
5	$[S \rightarrow AA \cdot, \$]$
6	$[A \rightarrow aA \cdot, a/b/\$]$ (merged I8+I10)

ACTION / GOTO table (s = shift, r1/r2/r3 = reduce by production 1/2/3, acc = accept):

State	a	b	\$	S	A
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

(v) Parsing Trace for the Input String 'aabb'

String “aabb” is a valid sentence: $S \Rightarrow A \ A \Rightarrow (aA) \ A \Rightarrow (a(aA))A \Rightarrow (a(a \ b))A \Rightarrow aab \cdot b$, i.e. the first $A = aab$ (via $A \rightarrow aA$, $A \rightarrow aA$, $A \rightarrow b$) and the second $A = b$ (via $A \rightarrow b$).

Step	Stack	Input Buffer	Action
1	0	a a b b \$	shift a $\rightarrow$ s3
2	0 a 3	a b b \$	shift a $\rightarrow$ s3
3	0 a 3 a 3	b b \$	shift b $\rightarrow$ s4
4	0 a 3 a 3 b 4	b \$	reduce r3: $A \rightarrow b$; GOTO(3,A)=6
5	0 a 3 a 3 A 6	b \$	reduce r2: $A \rightarrow aA$; GOTO(3,A)=6
6	0 a 3 A 6	b \$	reduce r2: $A \rightarrow aA$; GOTO(0,A)=2
7	0 A 2	b \$	shift b $\rightarrow$ s4
8	0 A 2 b 4	\$	reduce r3: $A \rightarrow b$; GOTO(2,A)=5
9	0 A 2 A 5	\$	reduce r1: $S \rightarrow AA$; GOTO(0,S)=1
10	0 S 1	\$	ACCEPT

Production used at each reduction: step 4 and step 8 use $A \rightarrow b$; steps 5 and 6 use $A \rightarrow aA$; step 9 uses $S \rightarrow AA$.

Verification: the stack reaches 0 S 1 exactly as the input is exhausted, and state 1 on lookahead \$ is the accept action, so the input string “aabb” is **ACCEPTED** by the LALR(1) parser. (As a simpler sanity check, the shorter string “bb” — parsed the same way via $A \rightarrow b$ twice and $S \rightarrow AA$ — is likewise accepted.)